

ASSIGNMENT

2110413 COMPUTER SECURITY

Assignment 2

Log Analysis

Computer Engineering, Chulalongkorn University

Worralop Srichainont

Student ID: 6632200221

August 31, 2026

Contents

1	Introduction	1
2	Part I: Suspicious Login Activity	1
2.1	Hackers	1
2.2	Attack Time	2
2.3	Busiest Server	3
2.4	Top Account	4
3	Part II: Sensitive Files on Web Servers	4
3.1	Sensitive Files	5
3.2	Targeted Files	7
4	Part III: Are There Bots Crawling Our Websites	7
4.1	Finding Bots	7
4.2	Bot Behavior	9

1 Introduction

For this assignment, we act as a member of the security/incident management team of a company named buttercup. The company's infrastructure consists of a mail server `mailsv` and three web servers `www1`, `www2`, `www3`. We are given the `secure.log` (authentication) and `access.log` (web access) files from these servers, and our job is to find out whether anyone is trying to hack, scan, or access protected information on them. Reading these logs line by line would be painful, so instead of writing our own parsing code, we use **Splunk** to search and aggregate the data.

We already imported the provided `tutorialdata.zip` into Splunk as a single upload (so every event is indexed exactly once), and every search below is run with the time range set to **All time**, since the events are not from the last 24 hours.

2 Part I: Suspicious Login Activity

On Linux servers, `secure.log` contains the authentication events, so this is where we look first to see if someone is trying to log in to a server they should not.

2.1 Q1: How Many Hackers, How Many Attempts

Definition: hacker and attempt

We define a **hacker** as a distinct source IP address that produced at least one **Failed password** line in `secure.log`. Each such line is counted as one **attempt**. This way, one hacker trying 200 times is still one hacker, but contributes 200 attempts.

To count them, we search for **Failed password** lines, extract the source IP with `rex`, then count the distinct IPs and the total number of matching lines.

Count Distinct Attackers

```
1 source="*secure.log" "Failed password"
2 | rex "from (?<src_ip>\S+) port"
3 | stats dc(src_ip) as distinct_hackers count as attempts
4
```

Figure 1 shows that there are **185** distinct source IPs, so we count **185 hackers** trying to get access to our servers, with **33,253 attempts** in total.

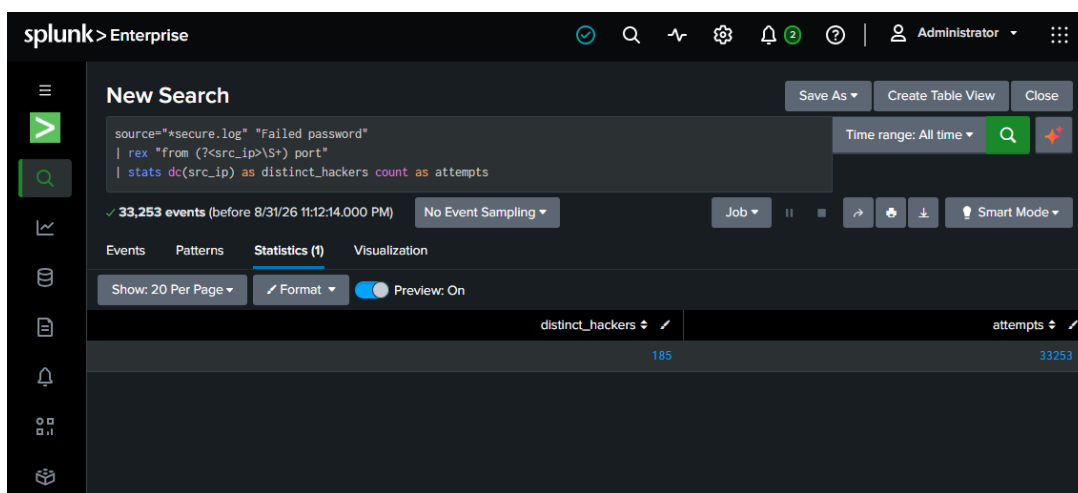


Figure 1: Distinct attacking IP addresses and total failed-password attempts.

2.2 Q2: What Time Do Hackers Appear

We bucket every Failed password event by hour of day and count how many attempts fall in each bucket.

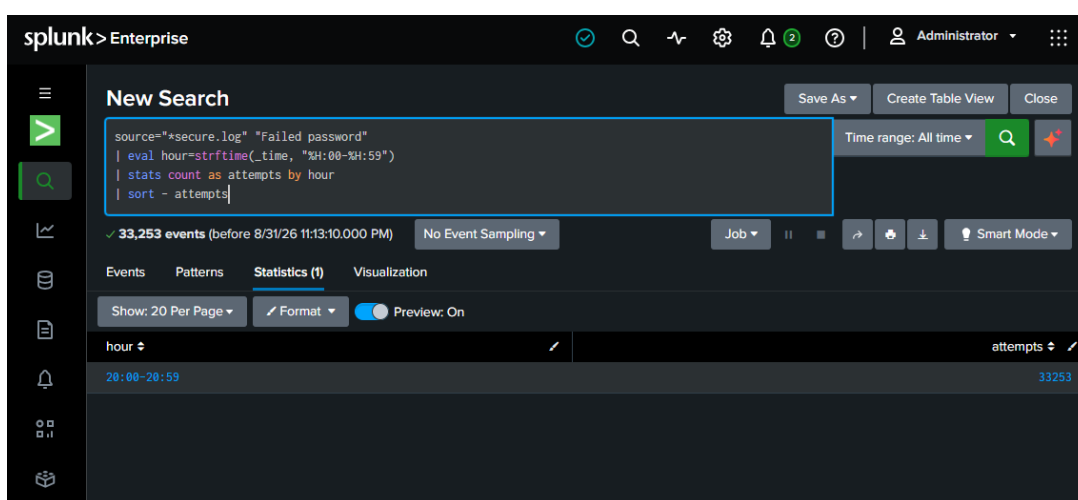
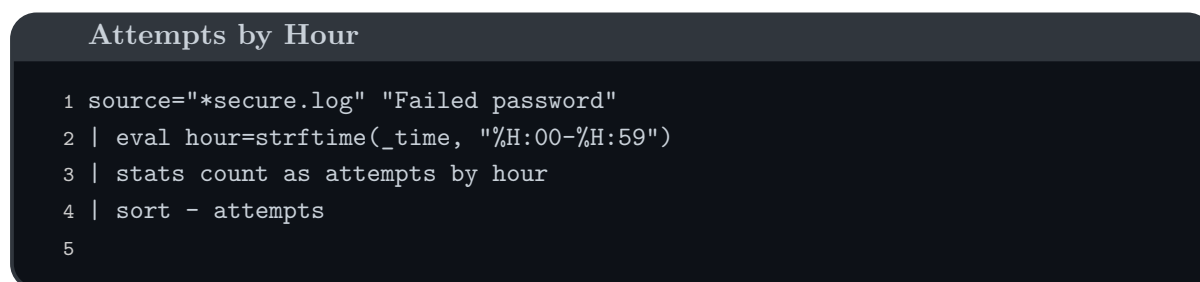


Figure 2: Failed-password attempts grouped by hour of day.

Figure 2 shows all 33,253 attempts falling into a single hour bucket, **20:00–20:59**. So, based on this data, hackers appear to try to hack our servers during the **20:00–20:59** hour.

2.3 Q3: Which Server Had the Most Attempts

Note: the `host` field is not reliable here

Our first instinct was to group by Splunk's `host` field, since we set the input to use "Segment in path" when uploading the data. This did not work as expected.

Attempts by Host (does not work)

```
1 source="*secure.log" "Failed password"
2 | stats count as attempts by host
3 | sort - attempts
4
```

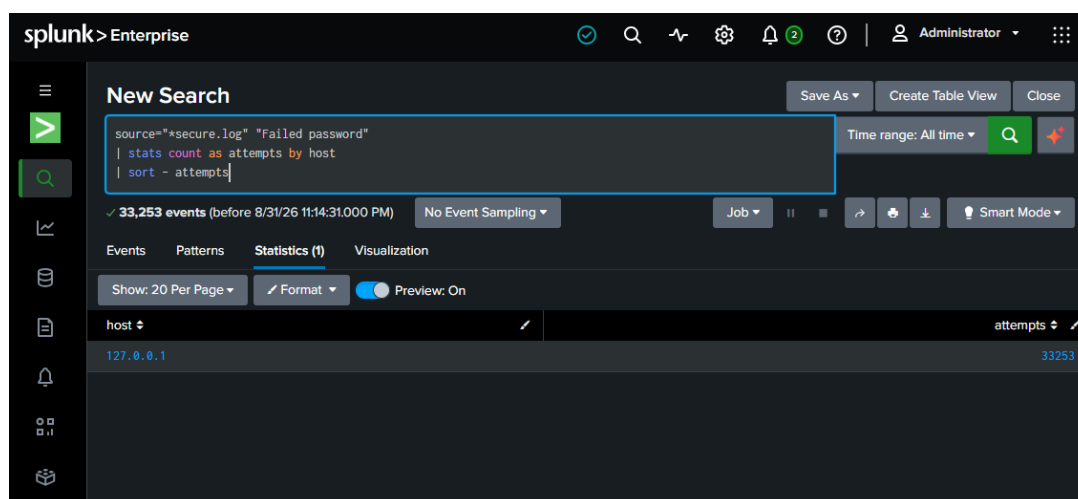


Figure 3: Grouping by `host` puts every event under a single `127.0.0.1` host.

Figure 3 shows all 33,253 events under one host, `127.0.0.1`, which is just the machine we uploaded the data from, not the actual server name. So instead of relying on `host`, we parse the server name straight out of the `source` file path with `rex`.

Attempts by Server

```
1 source="*secure.log" "Failed password"
2 | rex field=source "/(?<server>mailsv|www1|www2|www3)/secure\.log"
3 | stats count as attempts by server
4 | sort - attempts
5
```

Figure 4 shows `www1` with 8,798 attempts, ahead of `www3` (8,267), `mailsv` (8,154), and `www2` (8,034). So `www1` had the most attempts.

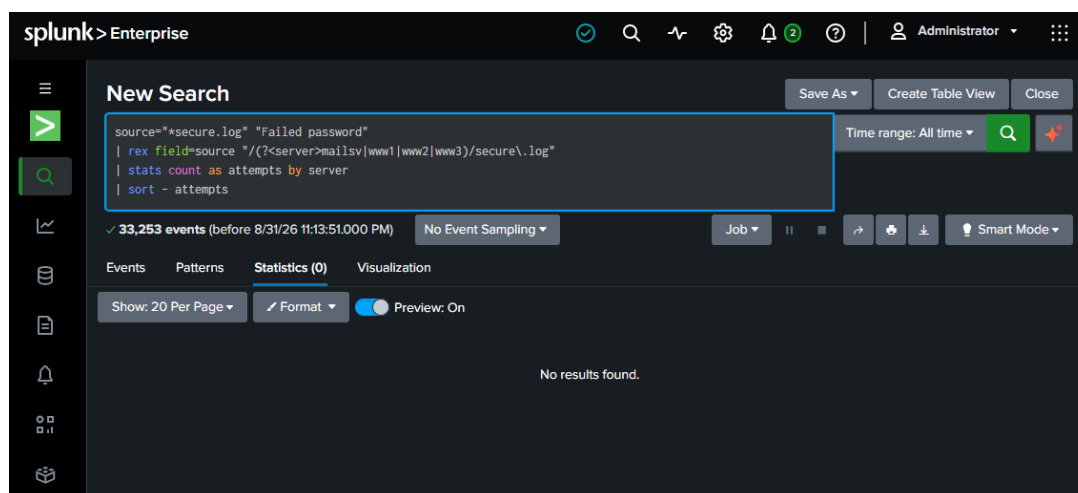


Figure 4: Failed-password attempts grouped by server, parsed from the source path.

Key Takeaway

www1 is the server under the heaviest brute-force pressure, but the difference across the four servers is small; attackers are not favoring one server much over the others.

2.4 Q4: Most Popular Account

We extract the account name from the same Failed password lines and take the top 5 by attempt count.

Top Attempted Accounts

```
1 source="*secure.log" "Failed password"
2 | rex "Failed password for (?<invalid user >)(?<account>\S+) from"
3 | stats count as attempts by account
4 | sort - attempts
5 | head 5
6
```

Figure 5 shows **root** as the most popular account (1,493 attempts), followed by administrator (1,020), admin (938), operator (923), and mail (753). Hackers are mostly guessing generic admin-sounding usernames rather than real people's accounts.

3 Part II: Sensitive Files on Web Servers

On web servers, `access.log` contains the web access events. Following the [security incident log review checklist](#), we look for excessive access attempts to non-existent files

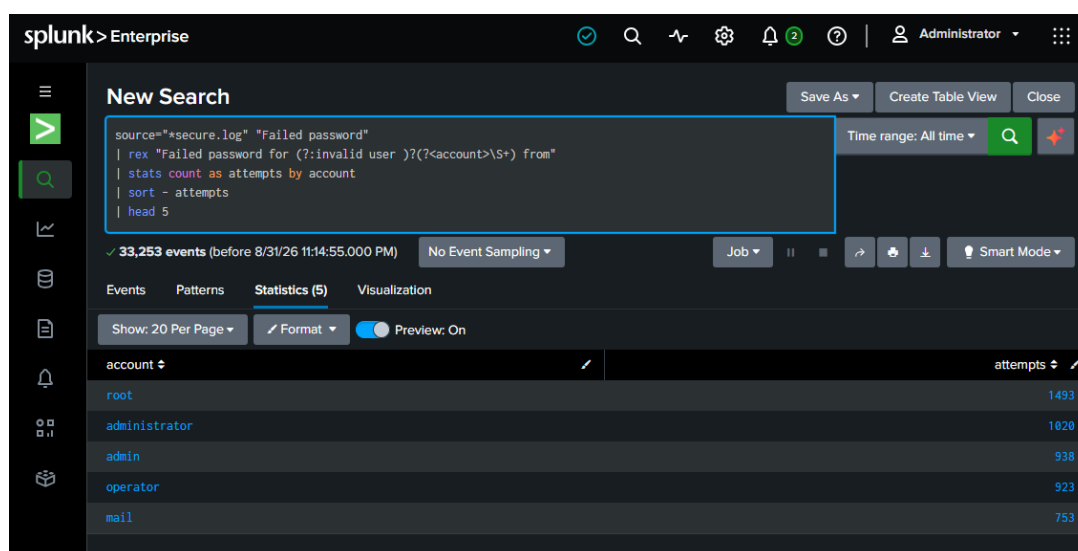


Figure 5: Top 5 account names used in failed-password attempts.

and error code 404 on files that are not part of our actual shop.

Note: what we are looking for

The checklist for web servers specifically calls out “excessive access attempts to non-existent files” and “access to extensions you have not implemented”. In this data set, we found four such paths, all returning **404**: `/passwords.pdf`, `/hidden/anna_nicole.html`, `/rush/signals.zip`, and `/numa/numa.html`. None of these belong to the real `buttercupgames.com` storefront, which only serves `/cart.do`, `/product.screen`, `/category.screen`, and similar pages.

3.1 Q5: Attempts to Access Sensitive Information

First, we count how many times any of the four suspicious paths were requested.

Count Sensitive-Path Probes

```
1 source="*access.log" ("/passwords.pdf" OR "/hidden/" OR "/rush/signals.zip"
  ↳ OR "/numa/numa.html")
2 | stats count as attempts
3
```

Figure 6 shows **284** attempts. As broader supporting evidence, we also look at every 404 response on the web servers, not just our four suspicious paths.

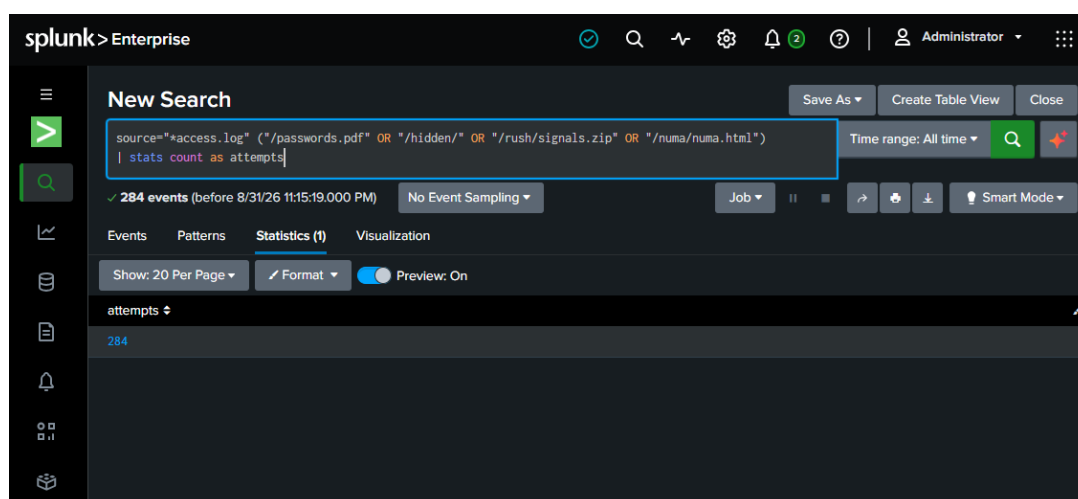


Figure 6: Total attempts to access the four suspicious paths.

All 404 Paths

```
1 source="*access.log" status=404
2 | stats count by uri_path
3 | sort - count
4
```

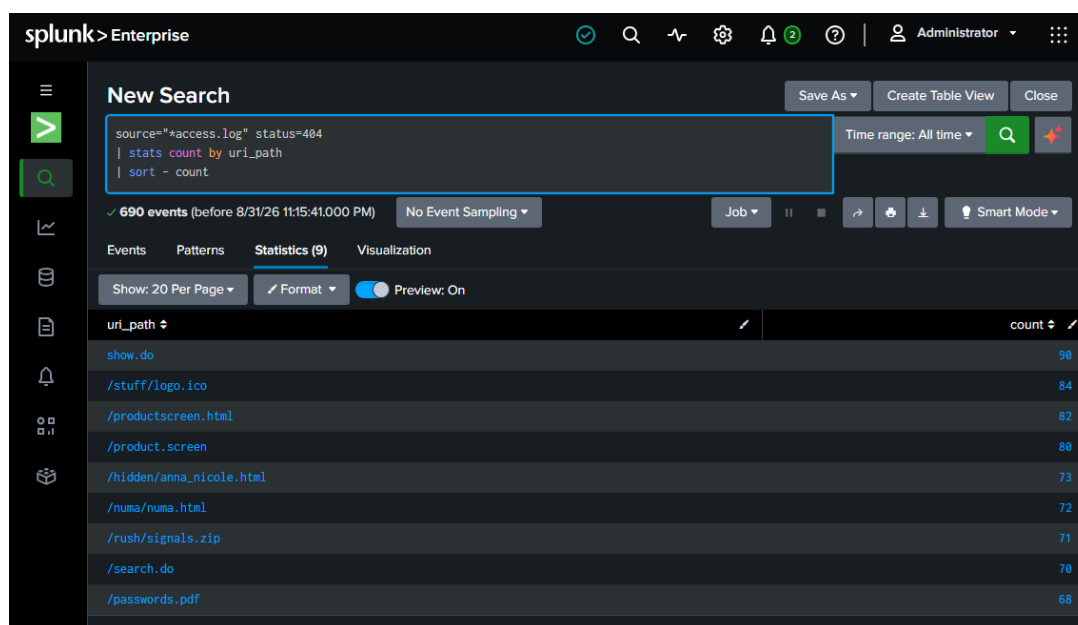


Figure 7: Every path that returned a 404, sorted by count.

Figure 7 shows 690 total 404 events across all paths, with our four suspicious paths (`/hidden/anna_nicole.html`, `/numa/numa.html`, `/rush/signals.zip`, `/passwords.pdf`) all sitting near the top of the list. So yes, we can find attempts to get access to sensitive information, and there were **284** such attempts.

3.2 Q6: What Resource Are Hackers Looking For

Sensitive-Path Probes by File

```
1 source="*access.log" ("/passwords.pdf" OR "/hidden/" OR "/rush/signals.zip"
  ↳ OR "/numa/numa.html")
2 | stats count as attempts by uri_path
3 | sort - attempts
4
```

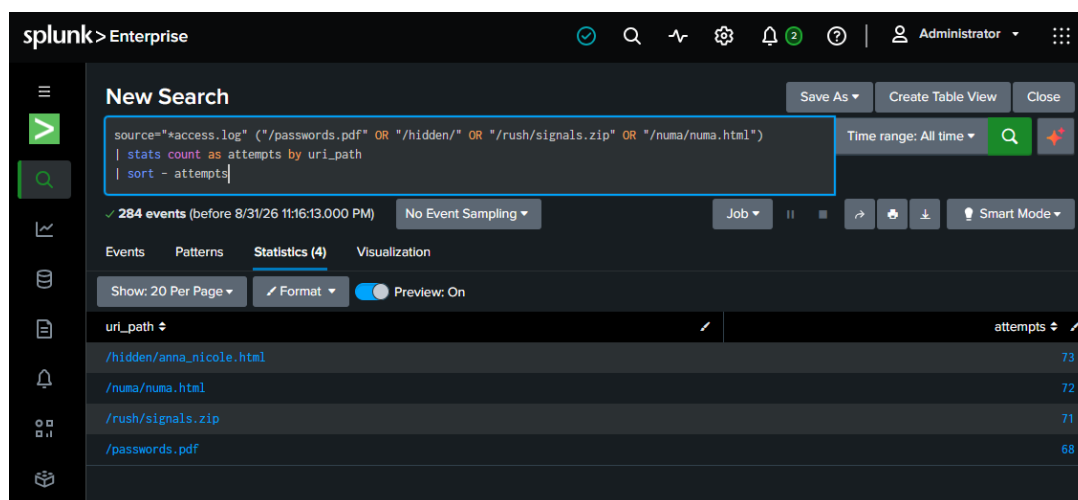


Figure 8: Attempts broken down by the specific file requested.

Figure 8 breaks the 284 attempts down by file: `/hidden/anna_nicole.html` (73), `/numa/numa.html` (72), `/rush/signals.zip` (71), and `/passwords.pdf` (68).

Key Takeaway

All four files are being probed at almost the same rate, but `passwords.pdf` is the clearest sensitive-information target by name; the others read more like an attacker fishing for any interesting-looking file on the server.

4 Part III: Are There Bots Crawling Our Websites

4.1 Q7: Finding Bots

The hint tells us to look at the `User-Agent` field in `access.log`. We first try filtering directly on Splunk's extracted `useragent` field.

Bots via useragent Field

```

1 source="*access.log" (useragent="*bot*" OR useragent="*Bot*" OR
  ↳ useragent="*crawler*" OR useragent="*Netcraft*")
2 | stats count as hits by useragent
3 | sort - hits
4

```

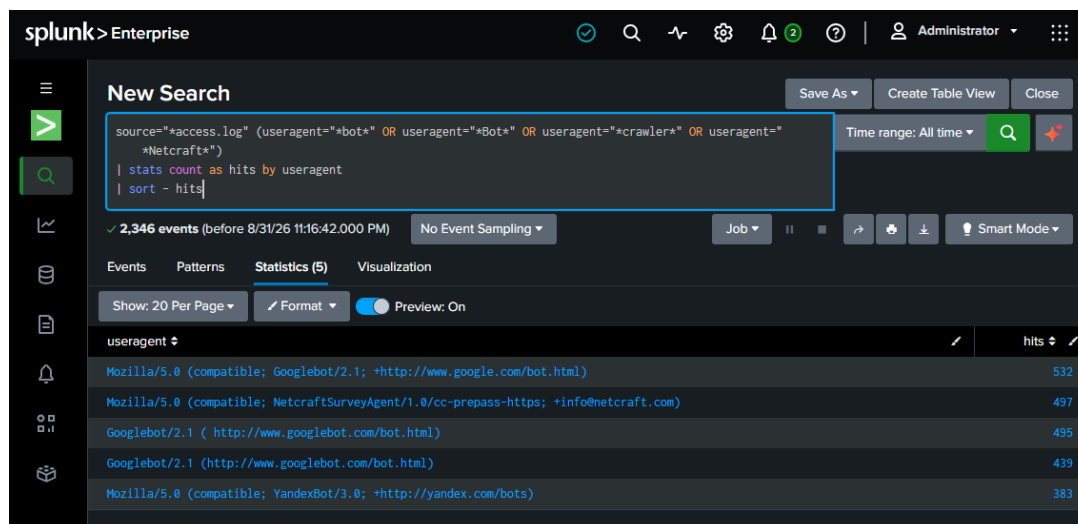


Figure 9: Requests grouped by bot-like user agent, using the extracted field.

As a cross-check, we also match on the raw event text and re-extract the user agent with `rex`, in case the `useragent` field was not extracted the same way for every event.

Bots via Raw-Text `rex` Fallback

```

1 source="*access.log" ("Googlebot" OR "YandexBot" OR "NetcraftSurveyAgent")
2 | rex "\"(?<ua>[^\"]+)"\s+\d+\s*$"
3 | stats count as hits by ua
4 | sort - hits
5

```

Both Figure 9 and Figure 10 agree on the same 2,346 events and the same top 5 user agents, so we are confident the field extraction is correct. So yes, we can find bots crawling our websites: **Googlebot** (three different user-agent strings, 532/495/439 hits), the **NetcraftSurveyAgent** (497 hits), and **YandexBot** (383 hits).

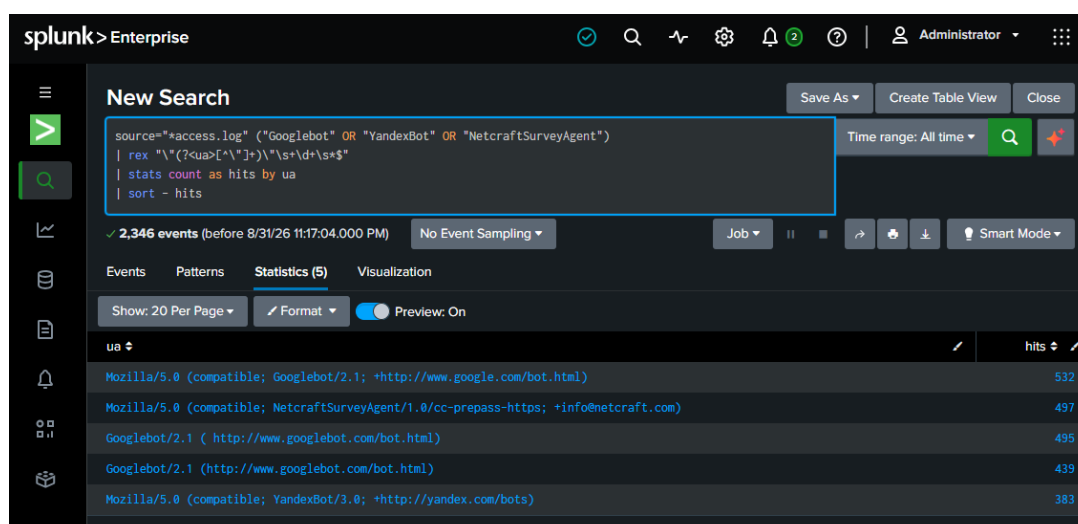


Figure 10: Same bot traffic, re-extracted directly from the raw event text.

4.2 Q8: What Are the Bots Doing

Bot Activity by Path and Status

```
1 source="*access.log" ("Googlebot" OR "YandexBot" OR "NetcraftSurveyAgent")
2 | stats count as hits by uri_path, status
3 | sort - hits
4
```

Figure 11 shows the bots mostly browsing the same storefront pages a normal shopper would: `/cart.do` (681), `/product.screen` (513), `/oldlink` (377), `/category.screen` (340), and `/cart/success.do` (125), almost all with status 200. Only a small number of hits land on error statuses (503, 500, 408, 406, 400) on those same pages.

Key Takeaway

The bot traffic we see (Googlebot, YandexBot, NetcraftSurveyAgent) is legitimate search-engine and monitoring crawling, not an attack; it walks the normal product catalog the same way a shopper would, unlike the sensitive-path probing in Part II.

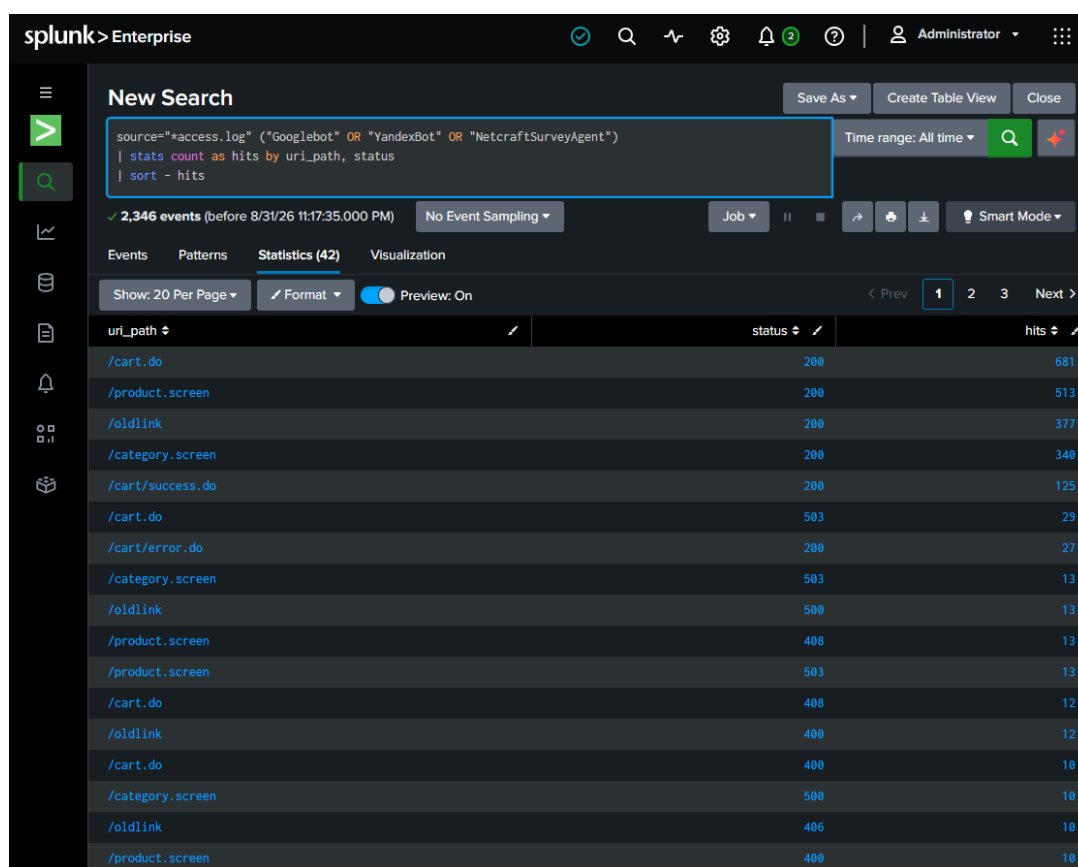


Figure 11: Bot traffic broken down by requested path and HTTP status.